Lecture 5 - Linear Models

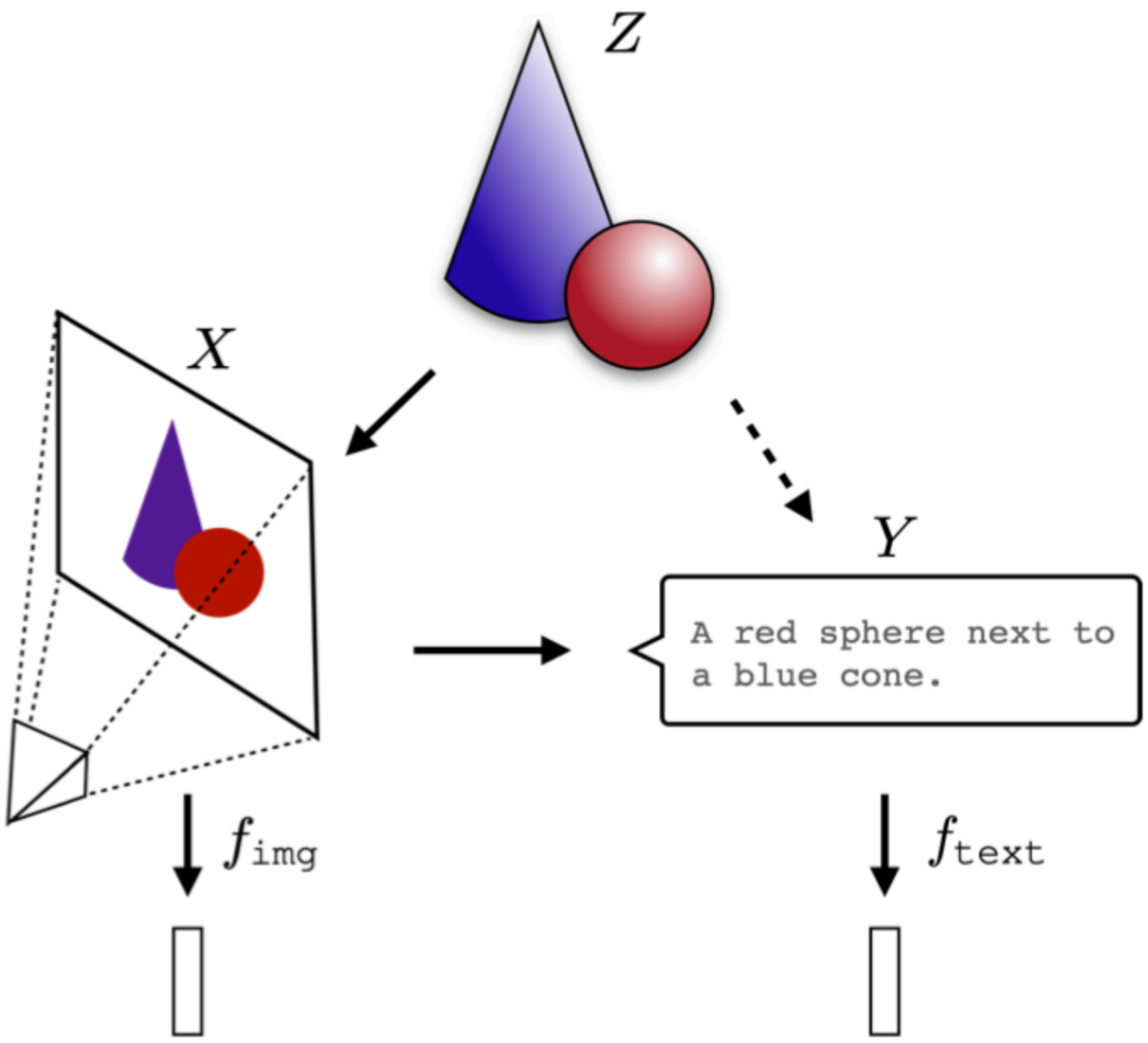
ECE 447: Data Analysis and Machine Learning for Engineers

Bahareh Tolooshams, Ph.D.
Assistant Professor,
University of Alberta
btolooshams@ualberta.ca
https://btolooshams.github.io

January 20, 2026

The Platonic Representation Hypothesis

Neural networks, trained with different objectives on different data and modalities, are converging to a shared statistical model of reality in their representation spaces.



This slide is informational, excluded from assessment.

Image and text from [1] Huh, Minyoung, et al. "Position: The platonic representation hypothesis." ICML 2024.

This Lecture Covers ...

- Linear Regression
  - Least squares
  - Gradient descent (very basic)

- Linear Classifier
  - Linear discriminant function
  - Minimum Euclidean distance (MED) classifier
  - Intro to minimum weighted Euclidean distance (WED) classifier

This Lecture Covers ...

- Linear Regression
  - Least squares
  - Gradient descent (very basic)

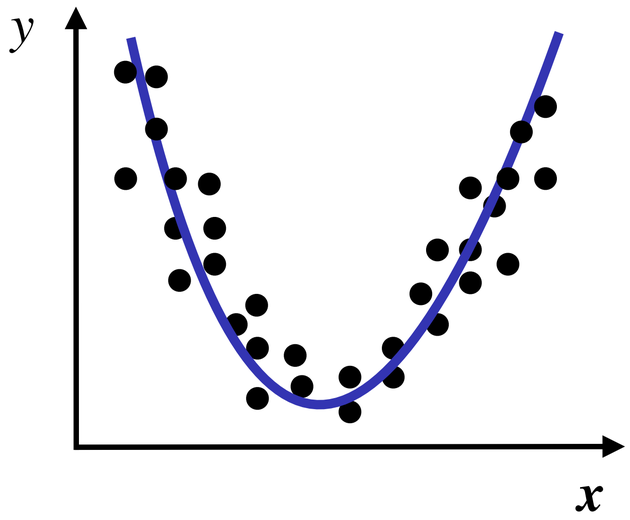
- Linear Classifier
  - Linear discriminant function
  - Minimum Euclidean distance (MED) classifier
  - Intro to minimum weighted Euclidean distance (WED) classifier

Curve Fitting Problem

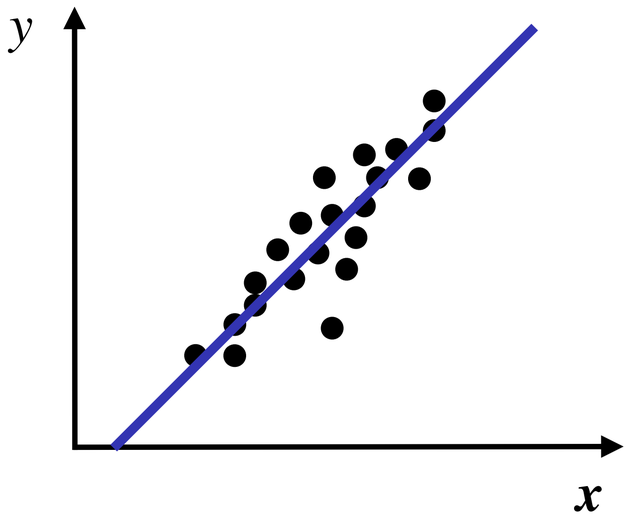
How to find a relation between $\boldsymbol{x}_i$ and y_i, such that for now data $\boldsymbol{x}_j$, I can predict y_j.

Given a set of data points $\{\boldsymbol{x}_i, y_i\}_{i=1}^{n}$,

One must find a curve or a surface that **fits** the data.



*Fit a **quadratic** function.*



*Fit a **linear** function.*

Which parametric functional form (model) is appropriate for the data at hand?

*It's important to **visualize** and **understand** the data prior to model selection.*

Finding the Optimal Parameters of the Fitting Model

Given a set of data points $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$, find the parameters $\boldsymbol{\theta}$ to fits the data.

$$y = f_{\boldsymbol{\theta}}(\boldsymbol{x})$$

Define a *non-negative loss* function

$$\mathscr{L}(\,\cdot\,,\,\cdot\,) : \mathbb{R} \times \mathbb{R} \to [0,\infty)$$

Compute $\boldsymbol{\theta}^*$ so as to minimize the total loss

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta} \in \mathscr{A}} \sum_{i=1}^{n} \mathscr{L}(y_i, f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))$$

The measured value of y_i.

The predicted one $\hat{y}_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$.

Linear Model with Square Loss - Deterministic Point of View

Given a set of data points $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$,

$$y = f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}_1^\top \boldsymbol{x} + \theta_0 \qquad \boldsymbol{\theta} = \begin{bmatrix} \boldsymbol{\theta}_1 \\ \theta_0 \end{bmatrix}$$

Define a *non-negative loss* function

$$\mathscr{L}(y, f_{\boldsymbol{\theta}}(\boldsymbol{x})) = (y - f_{\boldsymbol{\theta}}(\boldsymbol{x}))^2$$

Note. Nonlinear tasks can also be treated as a linear problem.

$$y = \theta_2 x^2 + \theta_1 x + \theta_0 = \theta_2 \phi_2(x) + \theta_1 \phi_1(x) + \theta_0$$

Model is linear with respect to $\phi(x) := \begin{bmatrix} x \\ x^2 \end{bmatrix}$.

Linear Regression — Model

Training set

Given a set of data points $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$,

$y_i \in \mathbb{R}$ output variable

$\boldsymbol{x}_i \in \mathbb{R}^p$ input vector (regressor)

The goal of the regression is to *estimate the parameter* $\boldsymbol{\theta}$ *from data modelled as*

$$y = f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}_1^\top \boldsymbol{x} + \theta_0 + \epsilon \qquad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

$$y = \boldsymbol{\theta}^\top \boldsymbol{x} + \epsilon$$

Absorb θ_0 into the parameter

$$\boldsymbol{\theta} = \begin{bmatrix} \boldsymbol{\theta}_1 \\ \theta_0 \end{bmatrix}$$

$$\boldsymbol{x} \leftarrow [\boldsymbol{x}, 1]^\top$$

Linear Regression — Loss Function

Given a set of data points $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$, minimize the following loss

$$\sum_{i=1}^{n} \mathscr{L}(y_i, f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) = \sum_{i=1}^{n} (y_i - \boldsymbol{\theta}^\top \boldsymbol{x}_i)^2.$$

To achieve this, we take the derivative of the loss with respect to $\boldsymbol{\theta}$ and set it to zero.

$$\nabla_{\boldsymbol{\theta}}(\sum_{i=1}^{n} \mathscr{L}(y_i, f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))) = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}}(\mathscr{L}(y_i, f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))) = 0$$

Take the Derivative and Set it to Zero to Find Optimal $\hat{\boldsymbol{\theta}}$

$$\sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}}((y_i - \hat{\boldsymbol{\theta}}^\top \boldsymbol{x}_i)^2) = 0$$

$$\sum_{i=1}^{n} 2(y_i - \hat{\boldsymbol{\theta}}^\top \boldsymbol{x}_i)\boldsymbol{x}_i = 0$$

$$\sum_{i=1}^{n} (\hat{\boldsymbol{\theta}}^\top \boldsymbol{x}_i)\boldsymbol{x}_i = \sum_{i=1}^{n} y_i \boldsymbol{x}_i$$

$$\sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^\top \hat{\boldsymbol{\theta}} = \sum_{i=1}^{n} y_i \boldsymbol{x}_i$$

$$\left(\sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^\top\right) \hat{\boldsymbol{\theta}} = \sum_{i=1}^{n} y_i \boldsymbol{x}_i$$

Note. This works because the loss function is convex. We will cover basics of convexity and derivatives in seminars.

For when $\boldsymbol{x}_i \in \mathbb{R}$ (scalar):

$$\hat{\boldsymbol{\theta}} = \frac{\sum_{i=1}^{n} y_i x_i}{\sum_{i=1}^{n} x_i^2}$$

Can We Write the Solution for Linear Regression in Matrix Form?

Lets put data into a vector $\boldsymbol{y}$ and matrix $\boldsymbol{X}$.

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n, \quad \boldsymbol{X} = \begin{bmatrix} - \boldsymbol{x}_1^\top - \\ - \boldsymbol{x}_2^\top - \\ \vdots \\ - \boldsymbol{x}_n^\top - \end{bmatrix} \in \mathbb{R}^{n \times p}$$

$$\boldsymbol{X}^\top \boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n] \begin{bmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} = \sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^\top$$

$$\boldsymbol{X}^\top \boldsymbol{y} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n] \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \sum_{i=1}^{n} y_i \boldsymbol{x}_i$$

$$\Rightarrow$$

$$(\boldsymbol{X}^\top \boldsymbol{X})\, \hat{\boldsymbol{\theta}} = \boldsymbol{X}^\top \boldsymbol{y}$$

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

the **Least Square (LS)** estimate.

Assuming, of course, that $(\boldsymbol{X}^\top \boldsymbol{X})^{-1}$ exists.

What if the Dataset is *too* Large for an Analytical Solution of Linear Regression?

We do optimization and gradient descent to minimize the loss.

Stochastic gradient descent

for i in range(n):

Here, we go over the data once! You usually will go over data more than once.

$$\text{loss} = (y_i - \boldsymbol{\theta}^\top \boldsymbol{x}_i)^2$$

$$\mathrm{g}^{(l)} = \nabla_{\boldsymbol{\theta}}\left((y_i - \boldsymbol{\theta}^\top \boldsymbol{x}_i)^2\right)$$

$$\boldsymbol{\theta}^{(l)} = \boldsymbol{\theta}^{(l-1)} - \alpha\ \mathrm{g}^{(l)}$$

We call this "we are training the model".

We are updating theta in a direction of reducing the loss in value.

This Lecture Covers ...

- Linear Regression
  - Least squares
  - Gradient descent (very basic)

- Linear Classifier
  - Linear discriminant function
  - Minimum Euclidean distance (MED) classifier
  - Intro to minimum weighted Euclidean distance (WED) classifier

Linear Models for Classification (linear discriminant function)

Given a set of training patterns $\{\boldsymbol{x}_i\}_{i=1}^n$ that belong to either of classes c_1 and c_2,

$$y_i = 1 \qquad \text{if } \boldsymbol{x}_i \in c_1$$
$$y_i = -1 \qquad \text{if } \boldsymbol{x}_i \in c_2$$

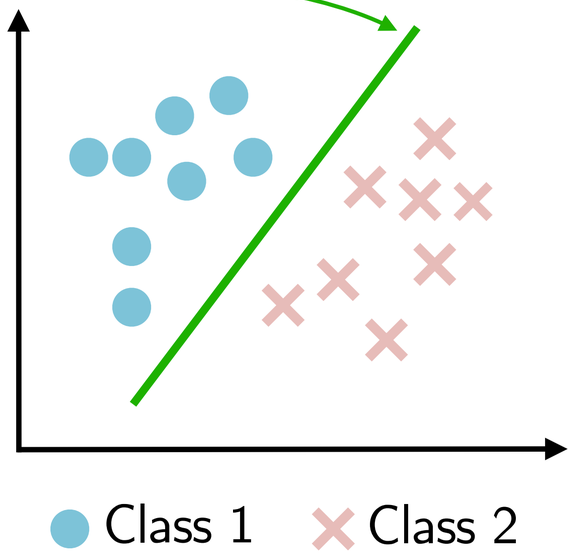
find a **hyperplane** (*line in 2-dim case*) that separates the two classes.

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}_1^\top \boldsymbol{x} + \theta_0$$

Decision boundary (sign): $\boldsymbol{x}_i \in c_1$ if $f_{\boldsymbol{\theta}}(\boldsymbol{x}) > 0$

$\boldsymbol{x}_i \in c_2$ if $f_{\boldsymbol{\theta}}(\boldsymbol{x}) < 0$

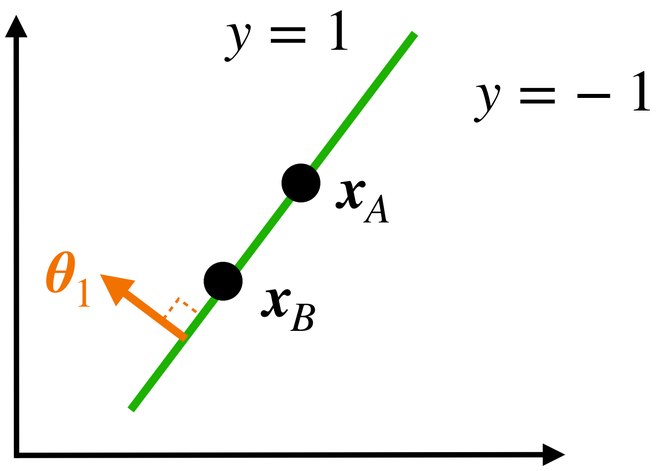
You are on the boundary $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = 0$



Visualizing Linear Discriminant Function

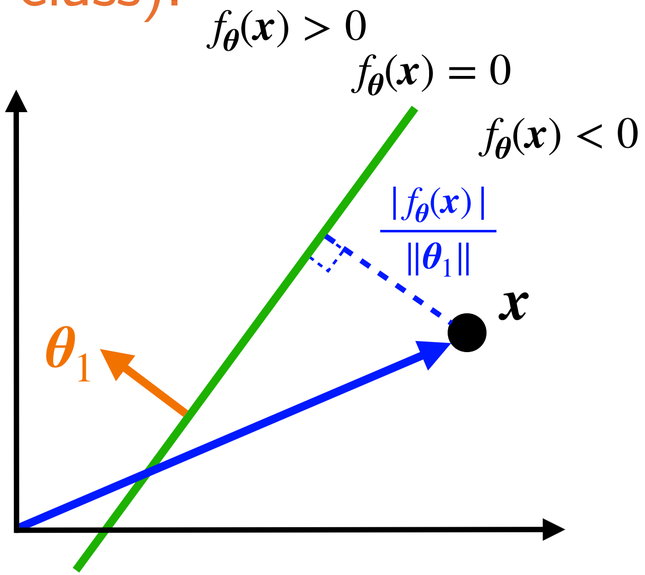
Consider two points $\boldsymbol{x}_A$ and $\boldsymbol{x}_B$ both of which lie on the decision surface, i.e., $f_{\boldsymbol{\theta}}(\boldsymbol{x}_A) = f_{\boldsymbol{\theta}}(\boldsymbol{x}_B) = 0$. Then,

$$\boldsymbol{\theta}_1^\top(\boldsymbol{x}_A - \boldsymbol{x}_B) = 0$$



$\boldsymbol{\theta}$ is perpendicular to the decision hyperplane (facing the + class).

Distance of a point $\boldsymbol{x}$ from the decision surface is $\dfrac{|f_{\boldsymbol{\theta}}(\boldsymbol{x})|}{\|\boldsymbol{\theta}_1\|}$.



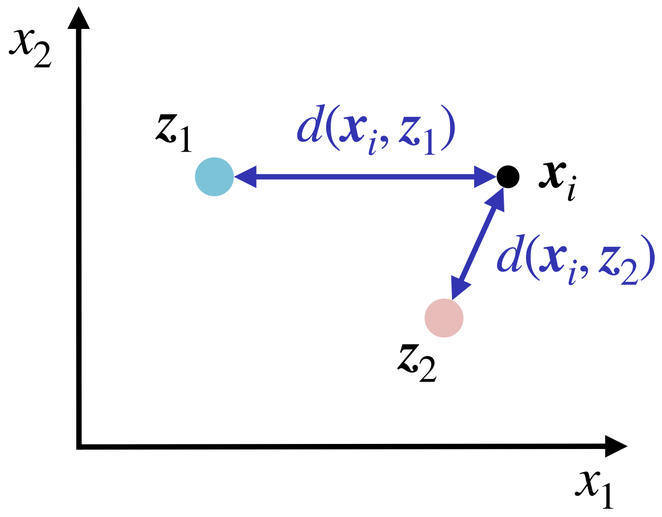
Linear Discriminant Function via Minimum Distance Classifier

Given a set of data $\{\boldsymbol{x}_i \in \mathbb{R}^p, y_i \in \{-1,1\}\}_{i=1}^n$, we define the classification decision as

$$\boldsymbol{x}_i \in c_k \text{ iff } d(\boldsymbol{x}_i, \boldsymbol{z}_k) < d(\boldsymbol{x}_i, \boldsymbol{z}_l) \qquad \text{for all } l \neq k$$

where $\boldsymbol{z}_k$ is a prototype of class c_k, and $d(\,\cdot\,,\cdot\,)$ is the distance function.

Intuition: Classify $\boldsymbol{x}_i$ to class c_1 if it is closer to the prototype of class c_1 than to c_2.



Optimization Objective for Minimum Distance Classifier

Another way to write the decision is via optimization, i.e., assign $\boldsymbol{x}$ to class c_k

$$k = \arg\min_j \; d(\boldsymbol{x}, \boldsymbol{z}_j)$$

Choose this as Minimum Euclidean Distance (MED)

$$f_k(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{z}_k\|_2^2$$

The decision boundary would be:

$$f(\boldsymbol{x}) = f_k(\boldsymbol{x}) - f_l(\boldsymbol{x})$$

The classification criteria is:

$$\boldsymbol{x} \in c_k \text{ iff } f_k(\boldsymbol{x}) < f_l(\boldsymbol{x}) \; \forall l \neq k$$

Decision Boundary for Minimum Euclidean Distance (MED) Classifier

The boundary is when $f_k(\boldsymbol{x}) = f_l(\boldsymbol{x})$ of two classes k and l. This gives

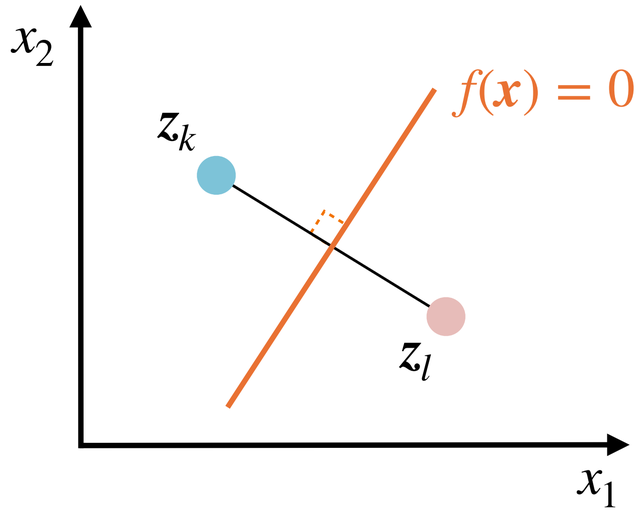
$$f(\boldsymbol{x}) = f_k(\boldsymbol{x}) - f_l(\boldsymbol{x}) = 0$$

$$\boldsymbol{x}^\top \boldsymbol{x} + z_k^\top z_k - 2z_k^\top \boldsymbol{x} - \boldsymbol{x}^\top \boldsymbol{x} - z_l^\top z_l + 2z_l^\top \boldsymbol{x} = 0$$

$$-z_k^\top \boldsymbol{x} + \frac{1}{2} z_k^\top z_k + z_l^\top \boldsymbol{x} - \frac{1}{2} z_l^\top z_l = 0$$

$$f(\boldsymbol{x}) = \underbrace{(z_l - z_k)^\top}_{\boldsymbol{\theta}_1} \boldsymbol{x} + \underbrace{\frac{1}{2}(z_k^\top z_k - z_l^\top z_l)}_{\theta_0} = 0$$

The MED decision boundary is perpendicular and in the middle of the line connecting z_k and z_l.



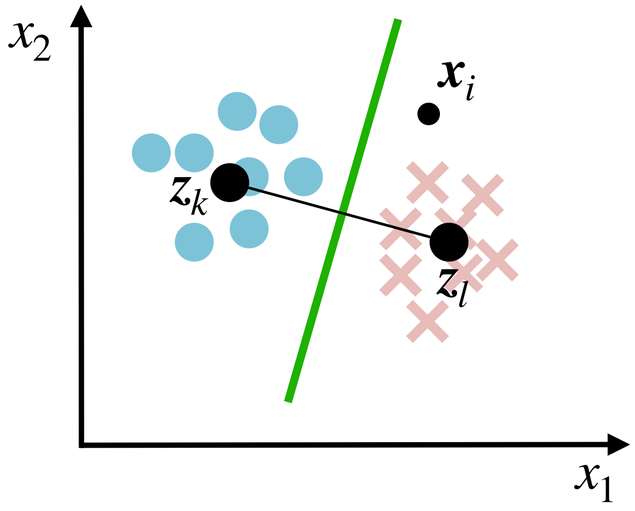
How to Choose the Prototype z_k for Each Class?

The choice of prototypes impact the behaviour of the classifier.

Ex. **Sample Mean**

Choose the empirical mean from each class as its prototype

$$z_k = \frac{1}{n_k} \sum_{i=1}^{n_k} \boldsymbol{x}_i \quad \text{for } \boldsymbol{x}_i \in c_k$$



Compare the new sample $\boldsymbol{x}_i$ to the mean of each class, and assign it to the one being closer.

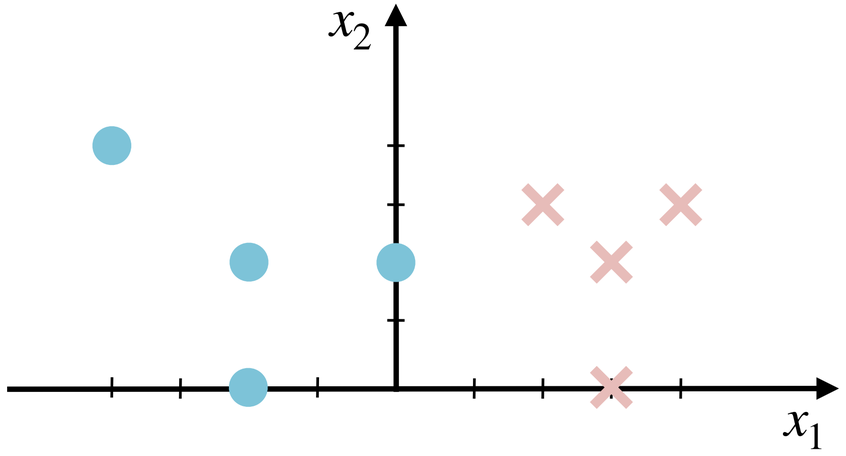
MED Classifier Example

Given a set of data,

$$\boldsymbol{x}_1 = [4\ 3]^\top, \boldsymbol{x}_2 = [3\ 0]^\top, \boldsymbol{x}_3 = [2\ 3]^\top, \boldsymbol{x}_4 = [3\ 2]^\top \text{ for class } c_1$$

$$\boldsymbol{x}_1 = [-2\ 2]^\top, \boldsymbol{x}_2 = [-2\ 0]^\top, \boldsymbol{x}_3 = [-4\ 4]^\top, \boldsymbol{x}_4 = [0\ 2]^\top \text{ for class } c_2$$

Compute and sketch the prototypes and the decision boundary.



MED Classifier Example - Compute the Sample Mean

$$z_1 = \frac{1}{n_1}\sum_{i=1}^{n_1} \boldsymbol{x}_i$$

$$= \frac{1}{4}([4\ 3]^\top + [3\ 0]^\top + [2\ 3]^\top + [3\ 2]^\top) \ = [3\ 2]^\top$$

$$z_2 = \frac{1}{n_2}\sum_{i=1}^{n_2} \boldsymbol{x}_i$$

$$= \frac{1}{4}([-2\ 2]^\top + [-2\ 0]^\top + [-4\ 4]^\top + [0\ 2]^\top) = [-2\ 2]^\top$$

MED Classifier Example - The Decision Boundary

$$f(\boldsymbol{x}) = (z_2 - z_1)^\top \boldsymbol{x} \;+\; \frac{1}{2}(z_1^\top z_1 - z_2^\top z_2) = 0$$

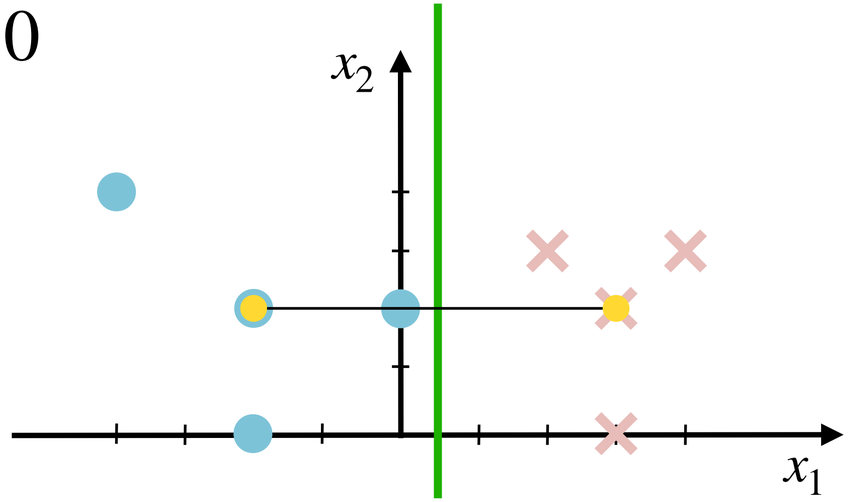
$$f(\boldsymbol{x}) = ([-2\ 2]^\top - [3\ 2]^\top)^\top \boldsymbol{x} \;+\; \frac{1}{2}([3\ 2]^\top [3\ 2] - [-2\ 2]^\top [-2\ 2]) = 0$$

$$f(\boldsymbol{x}) = [-5\ 0]\ \boldsymbol{x} \;+\; \frac{1}{2}(13 - 8) = 0$$

$$f(\boldsymbol{x}) = [-5\ 0] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 2.5 = 0$$

$$-5x_1 + 2.5 = 0$$

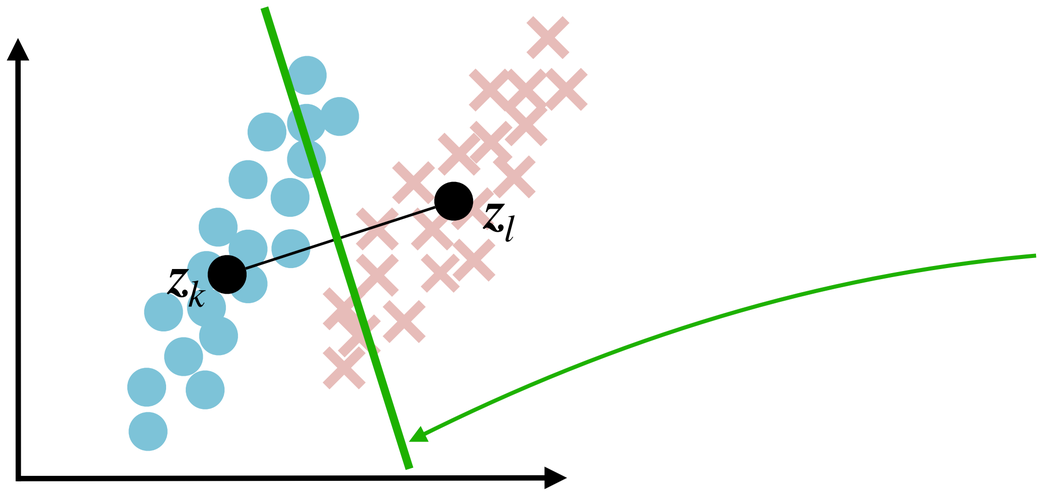
$$x_1 = 0.5$$



Advantages and Disadvantages of Sample Mean for MED

Advantages: You take the whole samples to compute the prototype. Hence, less sensitive to noise and outlier.

Disadvantages: Poor performance when distributions are not Gaussian-like clusters (e.g., thin or long clusters of samples)

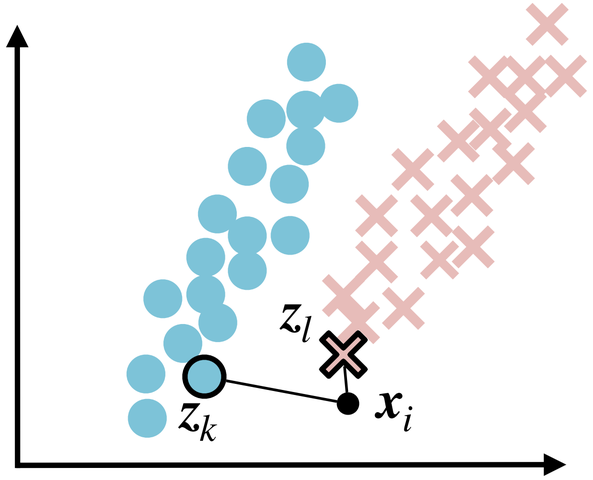


The classifier (decision boundary) has failed to properly separate the classes.

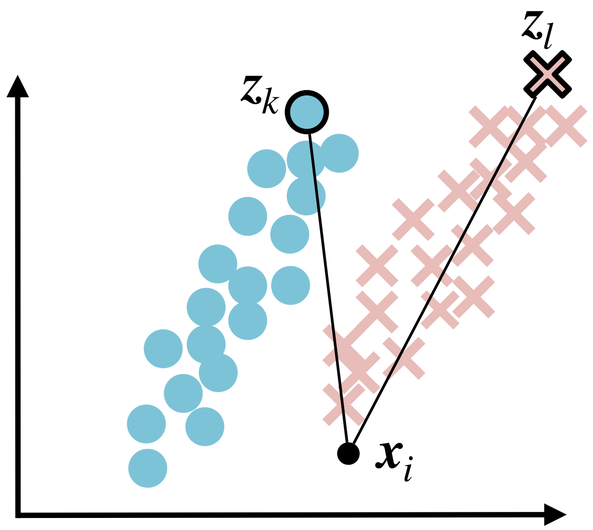
So, sample mean alone may not be a good prototype.

Other Examples of Prototypes — Similarity based Learning

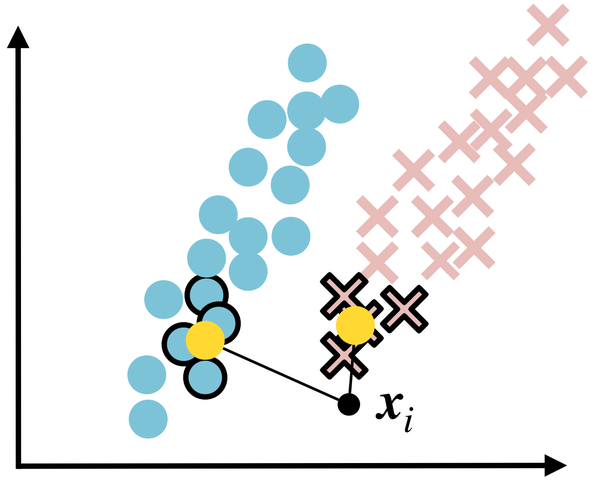
Nearest Neighbour (NN)



Farthest Neighbour

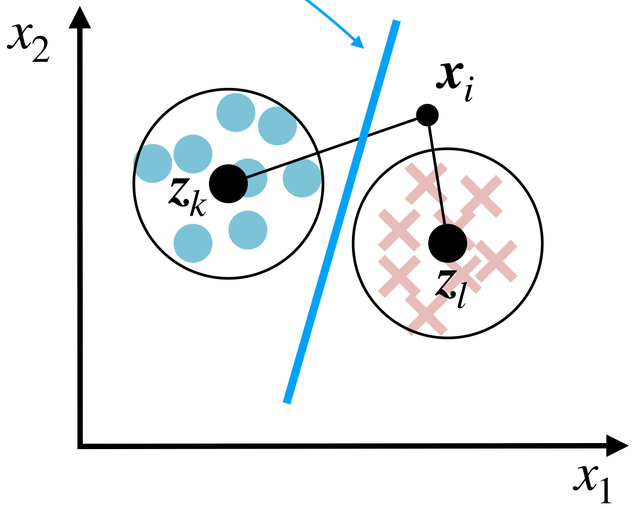


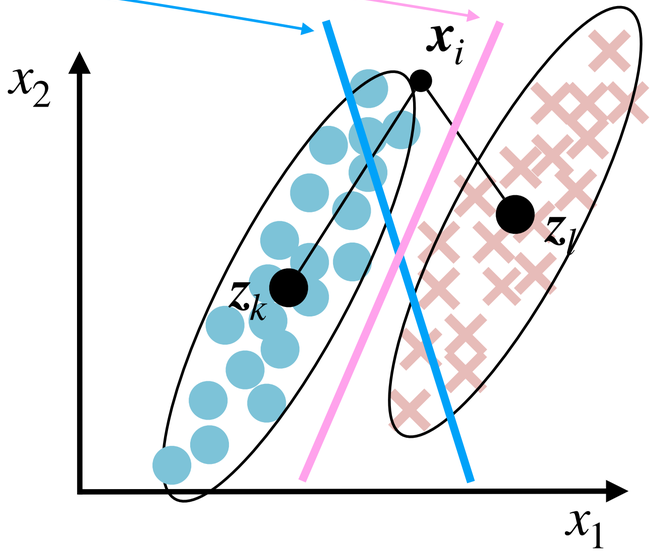
K-Nearest Neighbour (KNN)



This is the topic of a future lecture. We will re-visit later.

What to Do When Features are Correlated or at Different Scale?

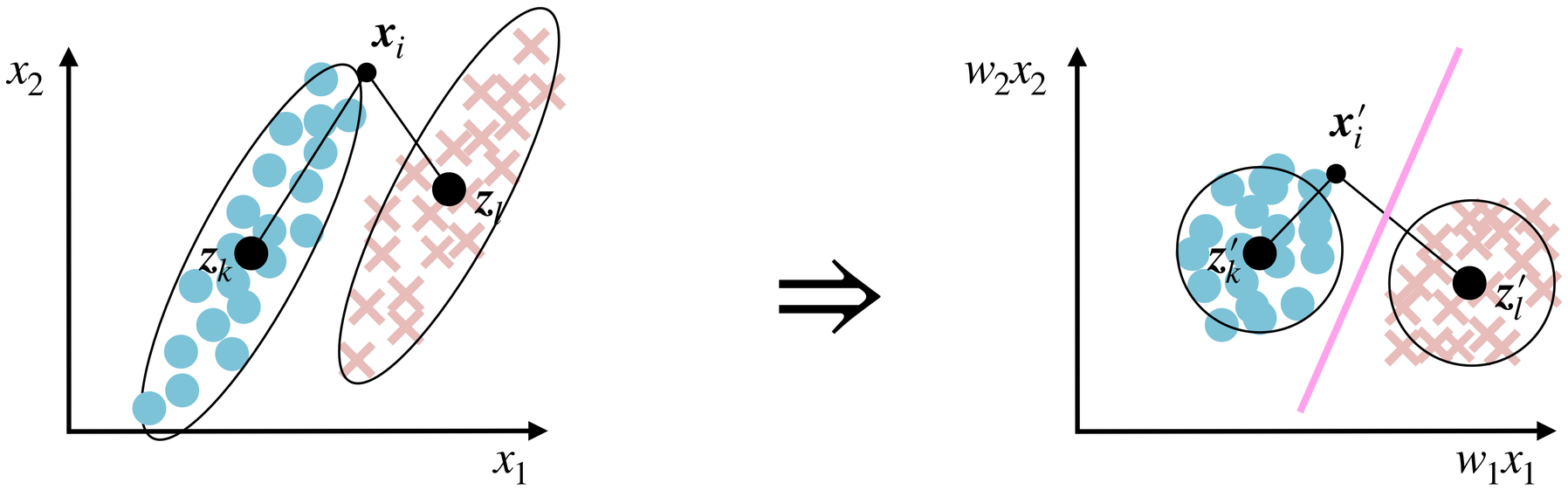




Model	Features are not much correlated The classes/features have similar unit/scale/variance	Features are correlated The features have different unit/scale/variance
Minimum Euclidean Distance	✔	✘
Minimum Weighted Euclidean Distance	✔	✔

Minimum **Weighted** Euclidean Distance (WED) Visualization

Note. This is for visualization purposes.



Intuition: Scale the features along with a linear transformation before applying the Euclidean distance metric.

Minimum Weighted Euclidean Distance

Euclidean Distance

$$d(\boldsymbol{x}, \boldsymbol{z}) = \left(\sum_{j=1}^{p} (x_j - z_j)^2 \right)^{\frac{1}{2}} = \|\boldsymbol{x} - \boldsymbol{z}\|_2$$

Weighted Euclidean Distance

$$d_{\boldsymbol{W}}(\boldsymbol{x}, \boldsymbol{z}) = \left(\sum_{j=1}^{p} w_j (x_j - z_j)^2 \right)^{\frac{1}{2}} = \|\boldsymbol{W}(\boldsymbol{x} - \boldsymbol{z})\|_2$$

$$= \left((\boldsymbol{x} - \boldsymbol{z})^\top \boldsymbol{W}^\top \boldsymbol{W} (\boldsymbol{x} - \boldsymbol{z}) \right)^{\frac{1}{2}}$$

$\boldsymbol{W}$: matrix with the weights w_j on its diagonal.

What Does the Form of This Equation Remind You of?

$$\left((\boldsymbol{x}-\boldsymbol{z})^\top \boldsymbol{W}^\top \boldsymbol{W}(\boldsymbol{x}-\boldsymbol{z})\right)^{\frac{1}{2}}$$

Topic of future lectures on a probabilistic perspective to machine learning

Decision Boundary for Minimum Weighted Euclidean Distance Classifier

Compute WED as $f_k(\boldsymbol{x})$ and $f_l(\boldsymbol{x})$ for each class to derivate the decision boundary.

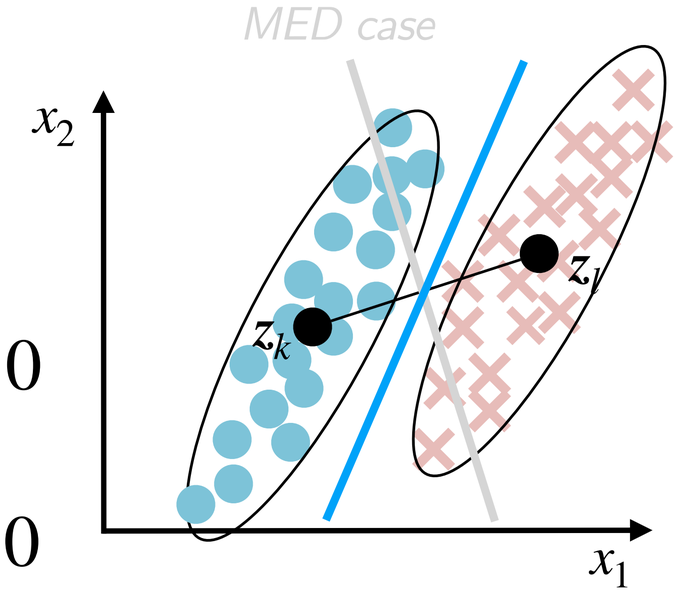
$$f(\boldsymbol{x}) = f_k(\boldsymbol{x}) - f_l(\boldsymbol{x}) = 0$$

$$\|\boldsymbol{W}(\boldsymbol{x} - \boldsymbol{z}_k)\|_2^2 - \|\boldsymbol{W}(\boldsymbol{x} - \boldsymbol{z}_l)\|_2^2 = 0$$

$$-\boldsymbol{z}_k^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{x} + \frac{1}{2}\boldsymbol{z}_k^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{z}_k + \boldsymbol{z}_l^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{x} - \frac{1}{2}\boldsymbol{z}_l^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{z}_l = 0$$

$$f(\boldsymbol{x}) = \underbrace{(\boldsymbol{z}_l - \boldsymbol{z}_k)^\top \boldsymbol{W}^\top \boldsymbol{W}}_{\boldsymbol{\theta}_1}\boldsymbol{x} + \underbrace{\frac{1}{2}(\boldsymbol{z}_k^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{z}_k - \boldsymbol{z}_l^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{z}_l)}_{\theta_0} = 0$$

Rotating the decision boundary.



Special case: assume same weight matrix $\boldsymbol{W}$ for both classes.

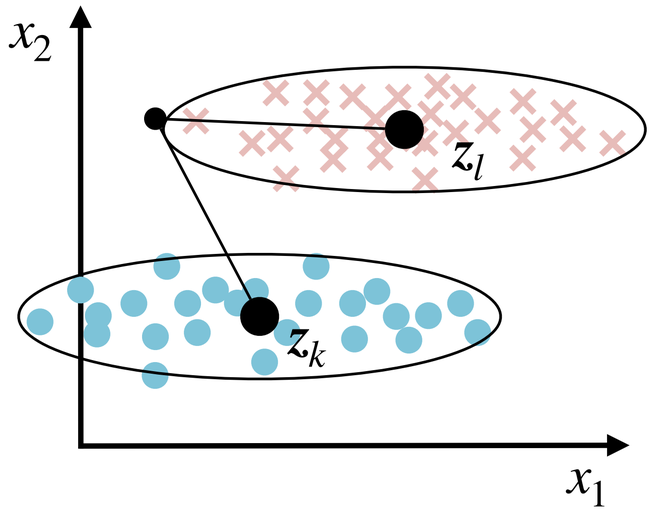
*Still **linear** classifier.*

Minimum Weighted Euclidean Distance Classifier Example

Given a set of data with following statistics

$$\boldsymbol{x} \sim \mathcal{N}\left(\begin{bmatrix}3\\3\end{bmatrix}, \begin{bmatrix}4 & 0\\0 & 1\end{bmatrix}\right) \text{ for } \boldsymbol{x} \in c_k$$

$$\boldsymbol{x} \sim \mathcal{N}\left(\begin{bmatrix}4\\5\end{bmatrix}, \begin{bmatrix}4 & 0\\0 & 1\end{bmatrix}\right) \text{ for } \boldsymbol{x} \in c_l$$



Derive the classifier for WED where $\boldsymbol{W}$ to weight each class is given by

$$\boldsymbol{W}_k = \begin{bmatrix}\frac{1}{2} & 0\\0 & 1\end{bmatrix} \qquad \text{and} \qquad \boldsymbol{W}_l = \begin{bmatrix}\frac{1}{2} & 0\\0 & 1\end{bmatrix}$$

WED Classifier Example - Compute the Decision Boundary

Choose the prototypes as the sample mean

$$z_1 = \frac{1}{n_1}\sum_{i=1}^{n_1} \boldsymbol{x}_i = \begin{bmatrix} 3 \\ 3 \end{bmatrix} \qquad z_2 = \frac{1}{n_2}\sum_{i=1}^{n_2} \boldsymbol{x}_i = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

The decision boundary is

$$f(\boldsymbol{x}) = (z_2 - z_1)^\top \boldsymbol{W}^\top \boldsymbol{W}\boldsymbol{x} + \frac{1}{2}(z_1^\top \boldsymbol{W}^\top \boldsymbol{W} z_1 - z_2^\top \boldsymbol{W}^\top \boldsymbol{W} z_2) = 0$$

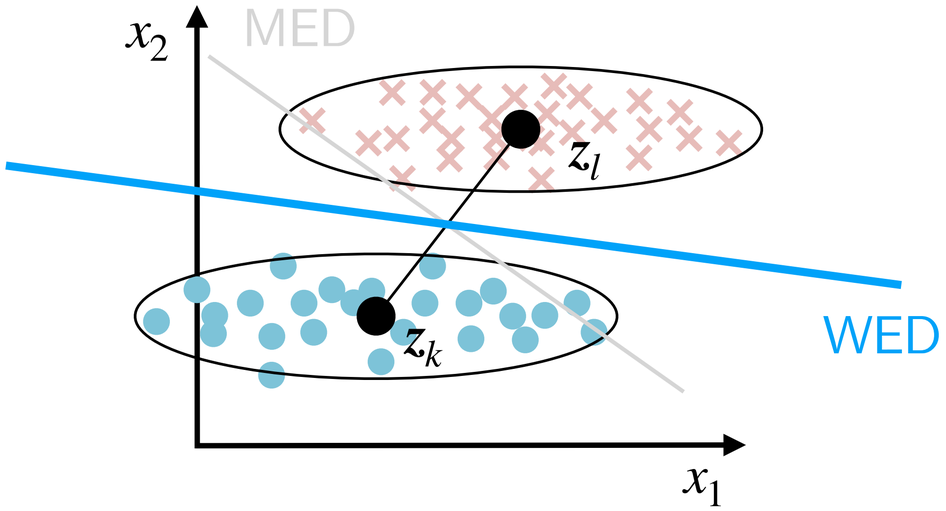
$$f(\boldsymbol{x}) = [1 \;\; 2] \begin{bmatrix} \frac{1}{4} & 0 \\ 0 & 1 \end{bmatrix} \boldsymbol{x} + \frac{1}{2}([3 \;\; 3] \begin{bmatrix} \frac{1}{4} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 3 \end{bmatrix} - [4 \;\; 5] \begin{bmatrix} \frac{1}{4} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \end{bmatrix}) = 0$$

WED Classifier Example - Decision Boundary Visualized

Note. This is for visualization purposes.

$$f(\boldsymbol{x}) = \begin{bmatrix} \frac{1}{2} & 2 \end{bmatrix} \boldsymbol{x} - 9.75 = 0$$

$$x_2 = -0.25x_1 + 3.875$$



Question: What if the $\boldsymbol{W}$ *matrix is* ***different*** *for each class? (future lectures)*

We will have a ***quadratic*** *classifier (not a line/surface anymore)!*

Two Ways to Think (Visualize in your Head) the WED Classifier

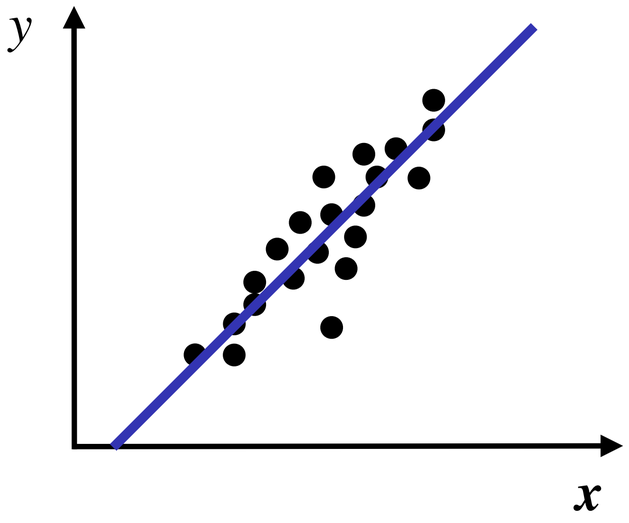
$$d_{\boldsymbol{W}}(\boldsymbol{x}, \boldsymbol{z}) = \left(\sum_{j=1}^{p} w_j (x_j - z_j)^2 \right)^{\frac{1}{2}}$$

(I) $\boldsymbol{W}$ *transforms the data (the distribution) into a hypersphere (e.g., rotate and scale the coordinate axes). Then, the classifier is linear using MED for decision boundary in that transformed space.*

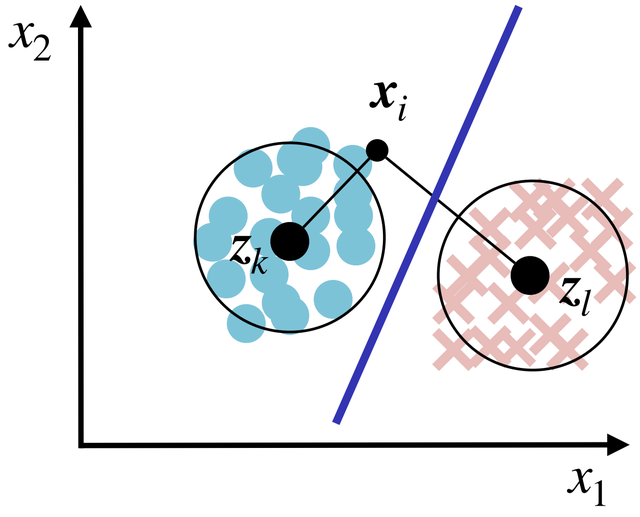
(II) $\boldsymbol{W}$ *transforms the decision boundary (rotate it or bend it) to capture the shape of the distribution.*

Machine Learning Algorithms

While machine learning method learn patterns from data, each still place a certain **assumption** (i.e., their **inductive bias**) on the data and use a mathematical formulation to classify/predict.



I want to find a linear model that minimizes the prediction error.



I want to classify my new point into the class whose prototype is closer to the point.

Takeaways

- ***Linear models*** — *Linear regression and least squares*

- ***Linear classifiers*** — *Decision boundary*

  — *Discriminant functions*

  — *Minimum Euclidean distance (MED)*

  — *Touch on minimum weighted Euclidean distance (WED)*